

Univerzitet u Novom Sadu
Tehnički fakultet „Mihajlo Pupin“
Zrenjanin

Akadska 2025/26. godina
Nastavni predmet: Arhitektura informacionih sistema

SEMINARSKI RAD

ODBC: pojam, uloga u razvoju informacionih sistema

Predmetni nastavnik:
Prof. dr Biljana Radulović

Predmetni asistent:
MSc Velibor Premčevski

Student:
Jovan Zec IT 09/21

Zrenjanin, 2026. godina

Sadržaj

1. Uvod.....	3
2. ODBC - pojam i osnovne karakteristike.....	3
2.1. Nastanak i standardizacija ODBC-a.....	3
2.2. Osnovne komponente ODBC arhitekture.....	3
2.3. DSN i connection string.....	4
3. Uloga ODBC-a u razvoju informacionih sistema.....	5
3.1. Nezavisnost od sistema za upravljanje bazom.....	5
3.2. Prenosivost, integracija i održavanje.....	5
3.3. Prednosti i ograničenja.....	6
4. Praktična implementacija - troslojna ASP.NET Core MVC aplikacija.....	7
4.1. Prezentacioni sloj.....	7
4.2. Business Logic Layer.....	7
4.3. Data Access Layer i ODBC komunikacija.....	8
4.4. Baza podataka i SQL Server.....	8
5. Funkcionalnosti aplikacije.....	9
6. Dalji koraci i mogućnosti unapređenja.....	9
7. Zaključak.....	10
8. Literatura.....	10

1. Uvod

Savremeni informacijski sistemi gotovo uvek rade sa podacima koji se čuvaju u relacionim ili drugim sistemima za upravljanje bazama podataka. Da bi aplikacija mogla da pristupi tim podacima, potreban je standardizovan mehanizam koji definiše način uspostavljanja konekcije, izvršavanja upita i preuzimanja rezultata. Jedan od najpoznatijih standarda koji rešava ovaj problem jeste ODBC (Open Database Connectivity).

ODBC predstavlja standardni programski interfejs za pristup bazama podataka. Njegova osnovna ideja je da aplikacija ne mora da poznaje unutrašnje detalje konkretnog sistema za upravljanje bazom podataka. Umesto direktne zavisnosti od SQL Server-a, MySQL-a, PostgreSQL-a ili drugog sistema, aplikacija komunicira preko ODBC interfejsa, dok odgovarajući ODBC drajver prevodi pozive u oblik koji konkretna baza razume.

Tema ovog rada je pojam ODBC-a i njegova uloga u razvoju informacijskih sistema. Pored teorijskog dela, izrađena je i demonstraciona server-klijent aplikacija za evidenciju studenata. Aplikacija je implementirana u programskom jeziku C# korišćenjem ASP.NET Core MVC tehnologije i .NET 8 platforme. Pristup SQL Server bazi realizovan je preko System.Data.Odbc biblioteke i Microsoft ODBC Driver 18 for SQL Server drajvera.

Aplikacija je organizovana primenom n-tier arhitekture sa jasnim razdvajanjem prezentacionog sloja, sloja poslovne logike i sloja za pristup podacima. Na taj način praktično se demonstrira kako ODBC može da bude smešten u Data Access Layer, dok ostatak sistema ostaje nezavisan od detalja konekcije sa bazom. Takva organizacija olakšava održavanje, testiranje i eventualnu zamenu tehnologije za skladištenje podataka.

2. ODBC - pojam i osnovne karakteristike

2.1. Nastanak i standardizacija ODBC-a

ODBC je skraćenica od Open Database Connectivity. Reč je o standardnom API-ju koji omogućava programima da pristupaju različitim izvorima podataka korišćenjem zajedničkog skupa funkcija. ODBC je nastao kao odgovor na problem velikog broja različitih baza podataka i proprietarnih načina povezivanja. Bez takvog sloja aplikacija bi morala da koristi poseban kod za svaki sistem baze podataka.

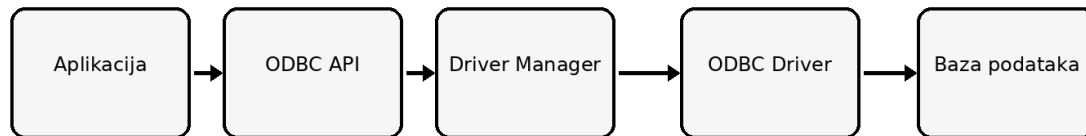
ODBC se zasniva na SQL standardima, ali ne pretpostavlja da sve baze podržavaju identične mogućnosti. Drajver ima zadatak da ODBC zahteve prevede na protokol i SQL dijalekt konkretnog sistema. Zbog toga isti koncept povezivanja može da se koristi sa različitim bazama, pod uslovom da za njih postoji odgovarajući ODBC drajver.

2.2. Osnovne komponente ODBC arhitekture

ODBC arhitekturu čine četiri osnovna elementa: aplikacija, Driver Manager, ODBC drajver i izvor podataka. Aplikacija generiše zahtev za pristup podacima. Driver Manager pronalazi i učitava odgovarajući drajver. ODBC drajver prevodi standardne ODBC pozive u komande

razumljive konkretnom sistemu baze, dok izvor podataka predstavlja samu bazu ili drugi sistem iz kog se podaci preuzimaju.

Tok ODBC komunikacije



SQL zahtev prolazi kroz standardizovani ODBC interfejs do odgovarajuceg drajvera, a rezultat se istim putem vraca aplikaciji.

Slika 1. Tok ODBC komunikacije

Driver Manager ima posebno važnu ulogu jer predstavlja posrednika između aplikacije i drajvera. Na Windows operativnom sistemu upravljanje ODBC izvorima podataka može se izvršiti pomoću alata ODBC Data Source Administrator. U njemu je moguće definisati korisničke, sistemske i fajl DSN konfiguracije, kao i proveriti instalirane drajvere.

2.3. DSN i connection string

DSN (Data Source Name) predstavlja imenovanu konfiguraciju koja sadrži informacije potrebne za povezivanje sa određenim izvorom podataka. U zavisnosti od načina primene, može se koristiti User DSN, System DSN ili File DSN. Prednost DSN pristupa je što se detalji konekcije mogu konfigurisati van izvornog koda aplikacije.

Drugi pristup je DSN-less konekcija, kod koje se parametri navode direktno u connection string-u. U demonstracionoj aplikaciji koristi se ovaj pristup. Primer connection string-a je: `Driver={ODBC Driver 18 for SQL Server};Server=localhost\SQLEXPRESS;Database=OdbcSeminarski;Trusted_Connection=Yes;TrustServerCertificate=Yes;`. Na ovaj način tačno je definisan drajver, SQL Server instanca, naziv baze i način autentifikacije.

Connection string u projektu nije postavljen unutar repozitorijumskog koda DAL sloja, već se čuva u konfiguracionom fajlu ASP.NET aplikacije. Time se postiže bolje razdvajanje odgovornosti. U produkcionom sistemu osetljivi podaci, kao što su korisničko ime i lozinka, ne bi trebalo da se čuvaju u javnom GitHub repozitorijumu, već u environment varijablama ili sistemu za upravljanje tajnama.

3. Uloga ODBC-a u razvoju informacionih sistema

3.1. Nezavisnost od sistema za upravljanje bazom

Jedna od najvažnijih uloga ODBC-a u razvoju informacionih sistema jeste smanjenje zavisnosti između aplikacije i konkretnog sistema za upravljanje bazom podataka. Aplikacija koristi standardizovan interfejs, dok drajver preuzima odgovornost za komunikaciju sa bazom. To ne znači da je svaka aplikacija automatski potpuno prenosiva između svih baza, jer se SQL dijalekti i podržane mogućnosti razlikuju, ali se način uspostavljanja konekcije i izvršavanja velikog broja operacija ujednačava.

Ova osobina je posebno korisna u informacionim sistemima koji moraju da se integrišu sa postojećim, često heterogenim izvorima podataka. Jedna organizacija može da koristi više sistema baza podataka, stare poslovne aplikacije ili specijalizovane izvore. Ako svaki od tih izvora nudi ODBC drajver, podaci se mogu uključiti u širi informacioni sistem bez razvoja potpuno novog komunikacionog protokola.

3.2. Prenosivost, integracija i održavanje

ODBC olakšava integraciju jer uvodi jasno definisanu granicu između aplikacionog koda i baze podataka. U n-tier arhitekturi ODBC kod prirodno pripada sloju za pristup podacima. Prezentacioni sloj ne zna da li DAL koristi ODBC, ORM ili neki drugi mehanizam. Sloj poslovne logike poziva interfejs repozitorijuma, a konkretna implementacija repozitorijuma komunicira sa bazom.

Takva organizacija poboljšava održavanje sistema. Ako dođe do promene connection string-a, drajvera ili načina izvršavanja SQL komandi, promene su uglavnom lokalizovane u DAL sloju. UI i poslovna pravila mogu da ostanu neizmenjeni. To je naročito važno kod većih informacionih sistema gde direktno mešanje SQL koda, korisničkog interfejsa i poslovne logike brzo dovodi do komplikovanog i teško održivog koda.

3.3. Prednosti i ograničenja

Glavne prednosti ODBC-a su standardizovan pristup podacima, veliki broj dostupnih drajvera, mogućnost povezivanja sa različitim izvorima, dobra podrška na Windows platformi i relativno jednostavno korišćenje u postojećim aplikacijama. ODBC je naročito koristan kada se zahteva direktna kontrola nad SQL upitima i kada nije potrebno koristiti pun ORM sloj.

Ograničenja ODBC-a proizilaze iz činjenice da je reč o opštem standardu. Specifične mogućnosti pojedinačnih baza mogu zahtevati dodatne prilagodbe, a razlike u SQL dijalektima i tipovima podataka i dalje postoje. Takođe, aplikacija zavisi od instalacije odgovarajućeg drajvera. Kod ručno pisanog SQL-a programer mora posebno da vodi računa o parametrizaciji upita, bezbednosti, transakcijama i mapiranju rezultata.

Prednosti	Ograničenja
Standardizovan API	Potreban odgovarajući drajver
Podrška za veliki broj baza	Razlike SQL dijalekata ostaju

Pogodan za integraciju starih sistema	Ručna obrada SQL-a i mapiranja
Nezavisnost viših slojeva aplikacije	Mogući problemi 32/64-bit drajvera
Mogućnost DSN i DSN-less povezivanja	Specifične funkcije baze nisu uvek prenosive

4. Praktična implementacija - troslojna ASP.NET Core MVC aplikacija

Za praktičnu demonstraciju izrađena je web aplikacija za evidenciju studenata. Korisnik može da pregleda postojeće zapise, doda novog studenta, pogleda detalje, izmeni podatke i obriše zapis. Cilj aplikacije nije kompleksnost funkcionalnosti, već jasan prikaz uloge ODBC-a u realnoj n-tier aplikaciji.

N-tier arhitektura seminarske aplikacije



Slika 2. N-tier arhitektura implementirane aplikacije

Solution sadrži tri projekta: OdbcIS.Web, OdbcIS.BLL i OdbcIS.DAL. Web projekat ima referencu samo na BLL projekat, dok BLL ima referencu na DAL. Time se poštuje pravilo da prezentacioni sloj ne pristupa bazi direktno. ODBC kod i entiteti baze nalaze se samo u DAL sloju.

4.1. Prezentacioni sloj

Prezentacioni sloj implementiran je kao ASP.NET Core MVC aplikacija. StudentsController je tanak kontroler: prihvata HTTP zahtev, proverava ModelState, mapira ViewModel u DTO i poziva IStudentService. Kontroler ne sadrži SQL upite niti direktnu komunikaciju sa bazom. Podaci se prema View-u prosleđuju isključivo preko StudentViewModel objekta.

ViewModel koristi DataAnnotations attribute za osnovnu validaciju korisničkog unosa, kao što su obavezna polja, format email adrese i dozvoljeni opseg godine studija. Interfejs je realizovan Razor View stranicama i Bootstrap stilovima. Kreirane su stranice Index, Create, Details, Edit i Delete.

4.2. Business Logic Layer

BLL je implementiran kao .NET Class Library. Glavnu ulogu ima StudentService koji primenjuje poslovna pravila nezavisno od korisničkog interfejsa. Pre upisa podataka normalizuju se ime, prezime, broj indeksa i email. Servis proverava validnost podataka i osigurava da broj indeksa bude jedinstven.

Komunikacija između Web sloja i BLL-a obavlja se preko StudentDto objekta i IStudentService interfejsa. Na taj način db entitet Student iz DAL sloja nikada se ne prosleđuje direktno u MVC View. Poslovna logika je centralizovana i mogla bi da se ponovo koristi iz drugog tipa klijenta bez izmene DAL-a.

4.3. Data Access Layer i ODBC komunikacija

DAL je takođe .NET Class Library i jedini je sloj koji direktno komunicira sa bazom. Sadrži Student entitet, OdbcConnectionFactory, IStudentRepository i StudentRepository. OdbcConnectionFactory kreira System.Data.Odbc.OdbcConnection koristeći connection string koji je prosleđen iz konfiguracije aplikacije.

StudentRepository sadrži parametrizovane SQL komande za SELECT, INSERT, UPDATE i DELETE operacije. ODBC koristi znak pitanja (?) kao placeholder za parametre. Za razliku od nekih drugih ADO.NET provajdera, kod ODBC-a redosled dodatih parametara mora odgovarati redosledu placeholder-a u SQL izrazu. Zato je u kodu posebno definisana pomoćna metoda koja parametre dodaje tačno određenim redosledom.

```
const string sql = @"  
INSERT INTO Students (FirstName, LastName, IndexNumber, Email, StudyYear)  
VALUES (?, ?, ?, ?, ?);";
```

```
await using var connection = _connectionFactory.CreateConnection();  
await connection.OpenAsync();  
await using var command = new OdbcCommand(sql, connection);
```

Parametrizovani upiti smanjuju rizik od SQL injection napada i istovremeno olakšavaju pravilno prosleđivanje različitih tipova podataka. Otvaranje konekcije obavlja se neposredno pre izvršavanja operacije, a await using konstrukcija obezbeđuje da se konekcija, komanda i reader pravilno oslobode i u slučaju izuzetka.

4.4. Baza podataka i SQL Server

Za demonstraciju je korišćen Microsoft SQL Server, a veza se ostvaruje preko Microsoft ODBC Driver 18 for SQL Server. Baza OdbcSeminarski sadrži tabelu Students sa kolonama Id, FirstName, LastName, IndexNumber, Email i StudyYear. Id je primarni ključ sa automatskim povećavanjem, dok IndexNumber ima UNIQUE ograničenje.

Repozitorijum sadrži SQL skriptu database/CreateDatabase.sql koja kreira bazu i tabelu i dodaje nekoliko početnih podataka. Time je pokretanje aplikacije na drugom računaru jednostavnije i reproduktivno. Connection string se po potrebi menja u appsettings.json, npr. ako SQL Server instanca nema naziv SQLEXPRESS.

5. Funkcionalnosti aplikacije

Početna stranica aplikacije prikazuje tabelu svih studenata. Podaci se dobijaju putem GET zahteva koji pokreće tok StudentsController -> StudentService -> StudentRepository -> ODBC konekcija -> SQL Server. Rezultat se mapira nazad u DTO, zatim u ViewModel i prikazuje u Razor View-u.

Funkcija dodavanja otvara formu za ime, prezime, broj indeksa, email i godinu studija. MVC prvo proverava validaciju ViewModel-a. Zatim BLL proverava poslovna pravila i jedinstvenost broja indeksa. Tek nakon uspešne validacije DAL izvršava parametrizovani INSERT upit preko ODBC-a.

Funkcija izmene koristi postojeći ID da učitava zapis, prikaže podatke i nakon potvrde izvrši UPDATE. Prilikom izmene BLL proverava da novi broj indeksa ne pripada drugom studentu. Funkcija brisanja najpre prikazuje stranicu za potvrdu, a zatim DAL izvršava DELETE upit. Stranica sa detaljima demonstrira pojedinačni SELECT upit sa parametrom.

Na ovaj način aplikacija demonstrira kompletan CRUD ciklus i pokazuje gde se ODBC nalazi u arhitekturi informacionog sistema. ODBC nije deo korisničkog interfejsa i ne predstavlja poslovnu logiku. Njegova uloga je infrastrukturna: omogućava DAL sloju da standardizovanim mehanizmom pošalje SQL komandu i dobije rezultat od baze.

Korisnička akcija	Sloj koji obrađuje	ODBC/SQL operacija
Prikaz liste	MVC -> BLL -> DAL	SELECT
Dodavanje	MVC -> BLL validacija -> DAL	INSERT
Detalji	MVC -> BLL -> DAL	SELECT WHERE Id = ?
Izmena	MVC -> BLL validacija -> DAL	UPDATE
Brisanje	MVC -> BLL -> DAL	DELETE

6. Dalji koraci i mogućnosti unapređenja

Demonstraciona aplikacija je namerno jednostavna kako bi fokus ostao na ODBC-u i n-tier arhitekturi. U narednoj verziji mogla bi se dodati autentifikacija i autorizacija korisnika, tako da samo ovlašćene osobe mogu menjati ili brisati podatke. Takođe bi bilo korisno dodati pretragu, sortiranje i straničenje za veći broj zapisa.

Sa aspekta ODBC-a, aplikacija bi mogla da se proširi izborom više izvora podataka. Na primer, jedna implementacija repozitorijuma mogla bi da koristi SQL Server, dok bi druga koristila drugi sistem sa ODBC drajverom. Time bi se još jasnije demonstrirala prednost standardizovanog interfejsa i zavisnost samo DAL sloja od konkretnog načina pristupa podacima.

Za produkciono okruženje bilo bi potrebno premestiti osetljive parametre konekcije iz običnog konfiguracionog fajla u environment varijable ili Secret Manager, dodati centralizovano logovanje grešaka, testove poslovne logike i transakcije za složenije operacije. Moguće je dodati i Docker okruženje za bazu, ali to nije neophodno za demonstraciju osnovne teme rada.

7. Zaključak

ODBC predstavlja važan standard za pristup podacima i ima značajnu ulogu u razvoju informacionih sistema, naročito kada je potrebno povezati aplikacije sa različitim ili postojećim sistemima baza podataka. Njegova glavna vrednost je u uvođenju standardizovanog interfejsa između aplikacije i specifičnog drajvera baze.

U praktičnom delu rada ODBC je primenjen u Data Access Layer-u ASP.NET Core MVC aplikacije za evidenciju studenata. Prezantacioni sloj oslanja se isključivo na BLL, BLL sadrži validaciju i tok poslovne logike, dok DAL sadrži entitete i direktnu ODBC komunikaciju sa SQL Server bazom. Time je postignuto jasno razdvajanje odgovornosti koje odgovara principima n-tier arhitekture.

Rad pokazuje da ODBC i dalje predstavlja praktičan mehanizam kada je potreban direktan, kontrolisan i prenosiv pristup podacima. Iako savremeni ORM alati nude viši nivo apstrakcije, razumevanje ODBC-a je korisno za integraciju sistema, rad sa legacy rešenjima i razumevanje osnovnog toka komunikacije između aplikacije i baze podataka.

8. Literatura

- [1] Microsoft Learn: ODBC overview and programming reference.
- [2] Microsoft Learn: System.Data.Odbc namespace (.NET).
- [3] Microsoft Learn: ASP.NET Core MVC overview.
- [4] Microsoft Learn: ODBC Driver 18 for SQL Server.
- [5] ISO/IEC 9075 - SQL standard (osnovni koncepti SQL pristupa podacima).

Prilog A - Struktura GitHub repozitorijuma

```
OdbcSeminarski/  
├── .gitignore  
├── README.md  
├── OdbcSeminarski.sln  
├── src/  
│   ├── OdbcIS.Web/  
│   ├── OdbcIS.BLL/  
│   └── OdbcIS.DAL/  
├── database/  
│   └── CreateDatabase.sql  
└── docs/  
    ├── Seminarski rad - ODBC - Jovan Zec.docx  
    └── Seminarski rad - ODBC - Jovan Zec.pdf
```